AULA 03: GATEWAY (ORQUESTRADOR) E CONTAINER COM DOCKER

Para referência futura dos comandos, baixe este PDF.

Todos os códigos desta aula estão disponíveis no seguinte repositório do GitHub: https://github.com/kelvinalmeida/Aula-Docker.

Instale o Docker Desktop pelo link:

https://www.docker.com/

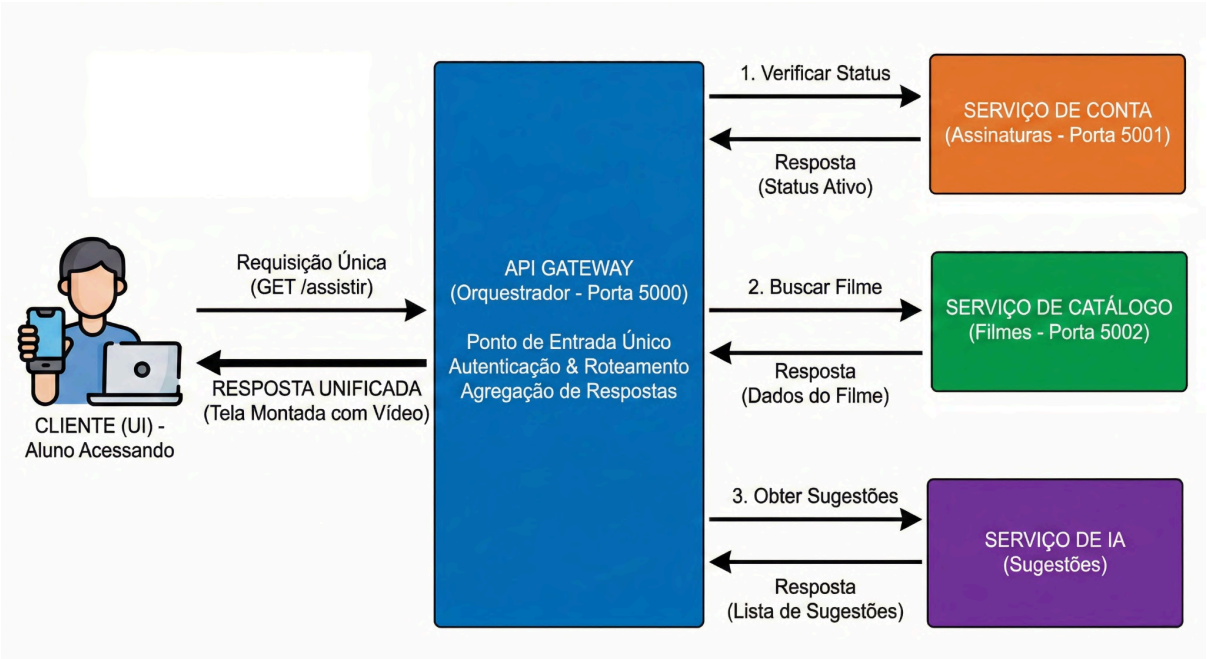
---

1. O Gateway (Orquestrador): O Maestro do Sistema

Imagine que o aluno não quer saber em qual porta o serviço de estoque ou pagamento está rodando; ele quer apenas acessar o site. O **Gateway** (ou Orquestrador) é o único ponto de entrada que recebe as requisições e as distribui para os serviços corretos.

- **Função:** Ele atua como um "escudo" e um "tradutor", protegendo os microsserviços internos e facilitando a vida do cliente (UI).

Analogia: *Imagine o Gateway como a Tela Inicial da Netflix. Quando o aluno abre o aplicativo, ele vê uma única tela montada perfeitamente. Mas, nos bastidores, o celular dele não foi buscar tudo em um lugar só.*



- A lista de filmes veio do Serviço de Catálogo.

- O nome "Status do usuario: ativo" veio do Serviço de Conta.
- A sugestão "Porque você assistiu X" veio do Serviço de IA.

O Gateway é o "porteiro" que recebe o clique do aluno, corre nos 3 serviços diferentes, pega as respostas, monta a página e entrega pronta. O aluno nem sabe que os outros serviços existem.

Exemplo Prático: "Mini-Netflix" (Código do Gateway, o orquestrador)

1. Serviço de Gateway (Porta 5000)

Neste código, o Gateway (Porta 5000) acessa os serviços de Filmes e Assinatura para liberar o play.

Salvar como: gateway_netflix.py

```
# gateway_netflix.py (Roda na porta 5000)
import requests
from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/')
def home():
    return "Bem-vindo ao Gateway da Netflix!"

@app.route('/assistir/<id_filme>/<id_usuario>')
def assistir_filme(id_filme, id_usuario):
    # return "Assistindo filme..."
    # 1. O Gateway pergunta ao Serviço de Assinaturas (Porta 5001) se o usuário
pagou
    # O aluno acessa a 5000, mas internamente o Gateway chama a 5001
    # usa o nome do serviço definido no docker
    resposta_conta =
requests.get(f'http://cont_servico_assinatura:5001/status/{id_usuario}').json()

    if resposta_conta['status'] != 'ativo':
        return jsonify({"erro": "Por favor, renove sua assinatura para
assistir."}), 403

    # 2. Se pagou, o Gateway busca o link do vídeo no Serviço de Catálogo (Porta
5002)
    resposta_filme =
requests.get(f'http://cont_servico_catalogo:5002/filme/{id_filme}').json()
```

```
    # 3. Entrega tudo pronto para o usuário
    return jsonify({
        "mensagem": f"Reproduzindo: {resposta_filme['titulo']}",
        "link_video": resposta_filme['url'],
        "qualidade": "4K"
    })

if __name__ == "__main__":
    app.run(port=5000)
```

2. Serviço de Assinaturas (Porta 5001)

Este serviço simula o banco de dados de clientes. Ele apenas responde se o usuário pagou ou não.

Salvar como: `servico_assinatura.py`

```
from flask import Flask, jsonify

app = Flask(__name__)

# Simulação de Banco de Dados de Usuários
USUARIOS = {
    "user1": {"nome": "João Silva", "status": "ativo"},
    "user2": {"nome": "Maria Souza", "status": "inativo"}, # Não pagou
    "user3": {"nome": "Pedro Santos", "status": "ativo"}
}

@app.route('/status/<id_usuario>')
def verificar_status(id_usuario):
    # Busca o usuário no dicionário; se não achar, retorna status
'desconhecido'
    usuario = USUARIOS.get(id_usuario, {"status": "desconhecido"})

    return jsonify({
        "id": id_usuario,
        "status": usuario['status']
    })

if __name__ == "__main__":
    # Importante: Rodar na porta 5001
    app.run(port=5001)
```

3. Serviço de Catálogo de Filmes (Porta 5002)

Este serviço detém as informações sobre os vídeos. Ele não sabe quem são os usuários, apenas entende de filmes.

Salvar como: `catalogo_service.py`

```
from flask import Flask, jsonify

app = Flask(__name__)

# Simulação de Banco de Dados de Filmes
FILMES = {
    "filme1": {
        "titulo": "A Origem (Inception)",
        "url": "http://video-server/inception.mp4",
        "categoria": "Sci-Fi"
    },
    "filme2": {
        "titulo": "O Poderoso Chefão",
        "url": "http://video-server/godfather.mp4",
        "categoria": "Drama"
    }
}

@app.route('/filme/<id_filme>')
def detalhes_filme(id_filme):
    filme = FILMES.get(id_filme)

    if filme:
        return jsonify(filme)
    else:
        return jsonify({"erro": "Filme não encontrado"}), 404

if __name__ == "__main__":
    # Importante: Rodar na porta 5002
    app.run(port=5002)
```

---

2. O Conceito de Container (Docker)

O Problema: O aluno criou o código acima e funciona no computador dele. Quando ele envia para o professor, não roda porque o professor tem uma versão diferente do Python ou faltou instalar uma biblioteca. Isso é o clássico *"Na minha máquina funciona"*.

A Solução (Container): Imagine um **Container de Navio**. Não importa se o navio carrega carros, bananas ou eletrônicos; a caixa de metal é padronizada. O guindaste consegue mover qualquer caixa sem saber o que tem dentro.

- O **Docker** cria essa "caixa" digital. Ele empacota o código + o Python + as bibliotecas.
- Você entrega a "caixa fechada" para o professor. Ele só precisa "abrir a caixa" (rodar o container) e vai funcionar exatamente igual, sem instalar nada extra.

---

3. Criando os containers no docker

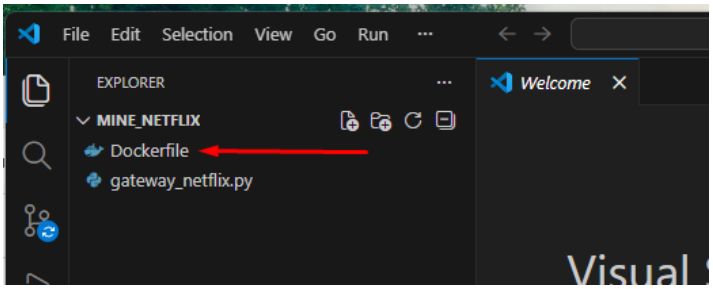
Obs: Você pode baixar o docker e aprender mais acessando o site: https://www.docker.com/

Criando a "Caixa" (O Dockerfile)

Para que o Gateway rode em qualquer computador sem erros, precisamos criar uma pasta chamada **mine_netflix** e criar um arquivo chamado **`Dockerfile`** (sem extensão) dentro da pasta e também colocar o código **gateway_netflix.py** dentro dessa pasta. Pense no **`Dockerfile`** como uma receita de bolo que o computador vai seguir.

A Receita do Gateway (Orquestrador)

Abaixo foi criado o arquivo chamado `Dockerfile` e `gateway_netflix.py` na mesma pasta chamada **mine_netflix**:



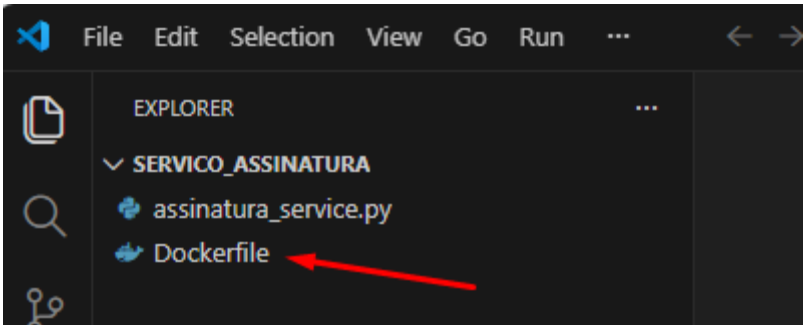
Dockerfile:

```
# 1. Escolha a base: Vamos usar um Python leve (versão 3.9 slim)
FROM python:3.9-slim

# 2. Defina o local de trabalho: Vamos criar uma pasta /app dentro da caixa
WORKDIR /app
```

```
# 3. Copie os ingredientes: Pega tudo da pasta atual e joga dentro da caixa
COPY . .

# 4. Prepare o ambiente: Instala o Flask e o Requests (as ferramentas
necessárias)
RUN pip install flask requests

# Definir a variável de ambiente para ativar o modo de desenvolvimento do
Flask
ENV FLASK_APP=gateway_netflix.py
ENV FLASK_RUN_HOST=0.0.0.0
ENV FLASK_RUN_PORT=5000
ENV FLASK_ENV=development

# Expor a porta padrão do Flask
EXPOSE 5000

# Rodar o aplicativo Flask
CMD ["flask", "run", "--host=0.0.0.0"]
```

A Receita do serviço de assinatura

Vamos fazer o mesmo para o serviço de assinatura. Abaixo foi criada a pasta **servico_assinatura** e criado o arquivo chamado `Dockerfile` e também colocado o arquivo `servico_assinatura.py` nessa pasta:



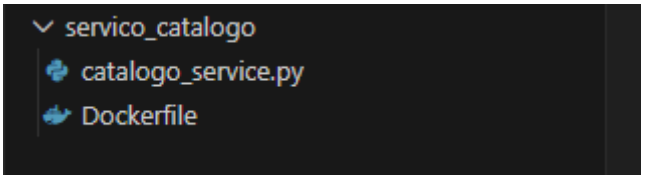
Dockerfile

```
# 1. Escolha a base: Vamos usar um Python leve (versão 3.9 slim)
FROM python:3.9-slim

# 2. Defina o local de trabalho: Vamos criar uma pasta /app dentro da caixa
```

```
WORKDIR /app

# 3. Copie os ingredientes: Pega tudo da pasta atual e joga dentro da caixa
COPY . .

# 4. Prepare o ambiente: Instala o Flask e o Requests (as ferramentas
necessárias)
RUN pip install flask requests

# Definir a variável de ambiente para ativar o modo de desenvolvimento do Flask
ENV FLASK_APP=servico_assinatura.py
ENV FLASK_RUN_HOST=0.0.0.0
ENV FLASK_RUN_PORT=5001
ENV FLASK_ENV=development

# Expor a porta padrão do Flask
EXPOSE 5001

# Rodar o aplicativo Flask
CMD ["flask", "run", "--host=0.0.0.0"]
```

A Receita do serviço de catálogo

Vamos fazer o mesmo para o serviço de catálogo. Abaixo foi criada a pasta **servico_catalogo** e criado o arquivo chamado `Dockerfile` e também colocado o arquivo `catalogo_service.py` nessa pasta:

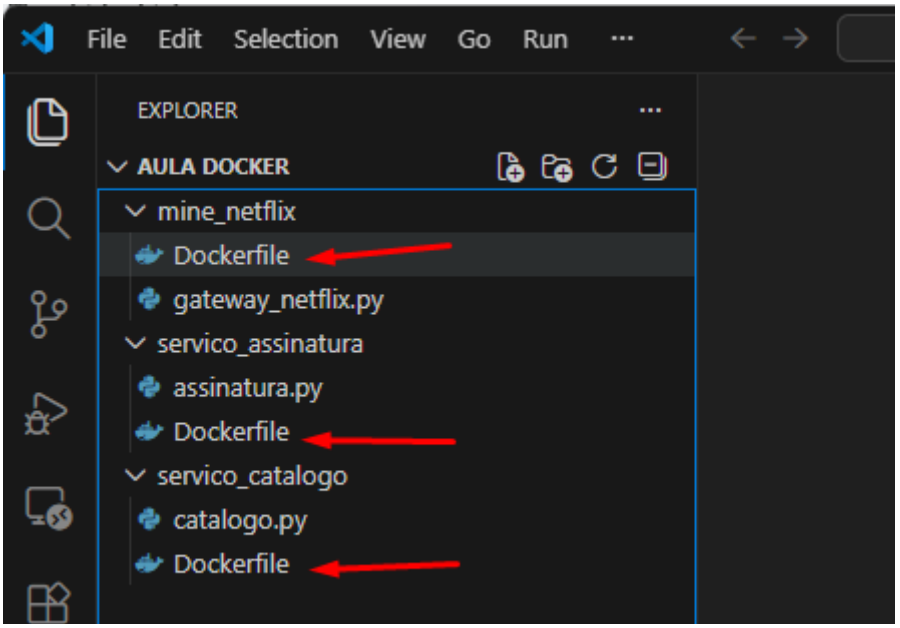


Dockerfile

```
# 1. Escolha a base: Vamos usar um Python leve (versão 3.9 slim)
FROM python:3.9-slim

# 2. Defina o local de trabalho: Vamos criar uma pasta /app dentro da caixa
WORKDIR /app

# 3. Copie os ingredientes: Pega tudo da pasta atual e joga dentro da caixa
COPY . .

# 4. Prepare o ambiente: Instala o Flask e o Requests (as ferramentas
necessárias)
```

```
RUN pip install flask requests

# Definir a variável de ambiente para ativar o modo de desenvolvimento do Flask
ENV FLASK_APP=catalogo_service.py
ENV FLASK_RUN_HOST=0.0.0.0
ENV FLASK_RUN_PORT=5002
ENV FLASK_ENV=development

# Expor a porta padrão do Flask
EXPOSE 5002

# Rodar o aplicativo Flask
CMD ["flask", "run", "--host=0.0.0.0"]
```

Ao final, você terá 3 pastas independentes, cada uma com sua própria "caixa" pronta para rodar!



---

4. Rodando os containers

Para transformar seu **Dockerfile** em um contêiner rodando na sua máquina, você geralmente precisa de dois comandos principais: um para **construir** a imagem e outro para **executar** o contêiner.

Aqui está o passo a passo direto:

1. Construir a Imagem (Build)

O primeiro passo é ler as instruções do arquivo **Dockerfile** e criar a "imagem" do sistema. Uma **imagem no Docker** é um pacote de software leve, auto suficiente e executável que inclui tudo o que é necessário para rodar uma aplicação: código, tempo de execução (runtime), bibliotecas, variáveis de ambiente e arquivos de configuração.

Pense nela como um **"snapshot" (instantâneo)** ou um **"blueprint" (projeto/planta)** do ambiente da sua aplicação.

É a partir dessa imagem que iremos criar o famoso container. Uma boa analogia para você entender ***imagem*** e ***container*** é: Pense na imagem como uma classe que contém todos os métodos e bibliotecas necessários, e o container é como se fosse uma instância dessa classe.

Para criar uma imagem utilizamos o comando:

```
docker build -t nome-da-minha-imagem .
```

- **docker build**: O comando de construção.
- **-t nome-da-minha-imagem**: O -t "tag" (nome) que você está dando para a imagem para facilitar o uso depois.
- **.** (ponto) Indica que o Dockerfile está na pasta atual.

- Como exemplo, para criar a imagem do serviço **mine_netflix**, que irá conter todo que é necessário para rodar esse serviço. Primeiro entre na pasta **mine_netflix** pelo terminal:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

v View in Docker Desktop    o View Config    w Enable Watch
PS C:\Users\kelvi\Desktop\Aula docker> cd .\mine_netflix\
PS C:\Users\kelvi\Desktop\Aula docker\mine_netflix>
```

e rode o seguinte comando no terminal:

```
docker build -t mine_netflix .
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

v View in Docker Desktop    o View Config    w Enable Watch
PS C:\Users\kelvi\Desktop\Aula docker> cd .\mine_netflix\
PS C:\Users\kelvi\Desktop\Aula docker\mine_netflix> docker build -t mine_netflix .
```

Após rodar o comando o Docker irá começar a criar a imagem chamada mine_netflix:

```
PS C:\Users\kelvi\Desktop\Aula docker\mine_netflix> docker build -t mine_netflix .
[+] Building 2.8s (9/9) FINISHED                                                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                0.0s
 => => transferring dockerfile: 748B                                                                                0.0s
 => [internal] load metadata for docker.io/library/python:3.9-slim                                                  2.5s
 => [internal] load .dockerignore                                                                                   0.0s
 => => transferring context: 2B                                                                                     0.0s
 => [1/4] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aab1acda1e13cf1731b1b  0.0s
 => => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aab1acda1e13cf1731b1b  0.0s
 => [internal] load build context                                                                                   0.0s
 => => transferring context: 163B                                                                                   0.0s
 => CACHED [2/4] WORKDIR /app                                                                                       0.0s
 => CACHED [3/4] COPY . .                                                                                           0.0s
 => CACHED [4/4] RUN pip install flask requests                                                                     0.0s
 => exporting to image                                                                                              0.1s
 => => exporting layers                                                                                             0.0s
 => => exporting manifest sha256:63219c1de3dd1a70a74072dc4379228d797c021c4c3906456879a4f601a6e2f5                   0.0s
 => => exporting config sha256:d73c51eb1b8de8d876015dfc9f61762ced35a7472b04dcf8e174cdda669decd5                     0.0s
 => => exporting attestation manifest sha256:9919ce634b3568cf2f5e3f93e9028a292a5a728e1cf05410286810ab859e76de       0.0s
 => => exporting manifest list sha256:738a45fb23c87c86e56ba52f085dd58789f1fcb8439eb06cce37a790841be8ca              0.0s
 => => naming to docker.io/library/mine_netflix:latest                                                              0.0s
 => => unpacking to docker.io/library/mine_netflix:latest                                                           0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/4ecgh8hy97qnxbd6sggmoo8c2
PS C:\Users\kelvi\Desktop\Aula docker\mine_netflix>
```

Se você rodar o comando “**docker images**” o Docker irá mostrar as imagens que você criou, por exemplo, você pode ver na figura abaixo a imagem **mine_netflix** que acabamos de criar:

```
PS C:\Users\kelvi\Desktop\Aula docker\mine_netflix> docker images
REPOSITORY              TAG       IMAGE ID       CREATED        SIZE
auladocker-assinatura   latest    aa3c1b49c1de   21 hours ago   208MB
auladocker-catalogo     latest    acf5c07de7d0   21 hours ago   208MB
auladocker-gateway      latest    0de69f344e1c   21 hours ago   208MB
mine_netflix            latest    738a45fb23c8   21 hours ago   208MB
servico_assinatura      latest    bdd0877400d5   44 hours ago   208MB
PS C:\Users\kelvi\Desktop\Aula docker\mine_netflix>
```

O mesmo você vai fazer para criar a imagem dos outros dois serviços: servico_de_assinatura e o servico_de_catalogo.

Para criar a imagem do serviço do serviço de assinatura entre na pasta “**servico_assinatura**” e rode o comando:

```
docker build -t servico_assinatura .
```

Para criar a imagem do serviço de catálogo entre na pasta “**servico_catalogo**” e rode o comando:

```
docker build -t servico_catalogo .
```

Após criar todos as imagens e rodar “**docker images**” no terminal veremos todas as imagens criadas:

```
PS C:\Users\kelvi\Desktop\Testando a aula docker> docker images
REPOSITORY          TAG       IMAGE ID       CREATED          SIZE
servico_catalogo    latest    03f94b9b3b72   20 minutes ago   208MB
servico_assinatura  latest    82709af6c998   20 minutes ago   208MB
mine_netflix        latest    e3bbd88929b1   21 minutes ago   208MB
PS C:\Users\kelvi\Desktop\Testando a aula docker>
```

Agora que já temos as imagens criadas, vamos criar os containers a partir dessas imagens.

---

5. Criando uma rede

É necessário criar uma rede para que os containers que criamos se comuniquem, pois cada container está isolado no seu próprio “mundo”, então é necessário criar uma rede.

Comando para criar uma Rede (Network)

No Docker, para que contêineres se chamem pelo nome, eles precisam estar na mesma rede virtual. O comando abaixo cria uma rede chamada **`netflix-rede:`**

```
docker network create netflix-rede
```

Após rodar o comando aparecerá um código, indicando que a rede foi criada.

```
PS C:\Users\kelvi> docker network create netflix-rede
386b0775c700618e838d7ec1726fc509215076955beec392cd481036240204c4
PS C:\Users\kelvi>
```

Para ver as redes criadas rodamos o comando:

```
docker network ls
```

Veja na imagem abaixo a rede que criamos:

```
PS C:\Users\kelvi> docker network ls
NETWORK ID     NAME           DRIVER    SCOPE
72f4ae7e89f1   bridge         bridge    local
8a91141e97c6   host           host      local
36698ad2852c   netflix-rede   bridge    local
24db715b79c1   none           null      local
```

---

6. Executar o Contêiner (Run)

Agora que as imagens e a rede existem, iremos transformá-lá em um processo ativo (contêiner):

Para transforma uma imagem em container usamos o seguinte comando:

```
docker run -d -p 8080:80 --network nome-da-rede --name nome-do-conteiner
nome-da-minha-imagem
```

- **-d**: (Detached) Roda o contêiner em segundo plano para não travar seu terminal.
- **-p 8080:80**: Mapeia as portas. O formato é porta-do-seu-pc:porta-dentro-do-docker.
- **--network**: nome da rede que será utilizada pelo container
- **--name**: Dá um nome amigável ao contêiner que está rodando.
- **nome-da-minha-imagem**: O nome da imagem que você quer criar o container.

Os comandos abaixo criam os containers das imagens que criamos:

1. Criando o container do serviço servico_assinatura

```
docker run -d -p 5001:5001 --network netflix-rede --name
cont_servico_assinatura servico_assinatura
```

O código abaixo mostra o que após rodar o comando aparece um código informando que o container foi criado e está rodando:

```
PS C:\Users\kelvi> docker run -d -p 5001:5001 --network netflix-rede --name cont_servico_assinatura servico_assinatura
5916686acfcfa428d2b8a3a61dfd8b9e1d16fa959058999b0a797fc0839cfd51
PS C:\Users\kelvi>
```

Ao rodarmos "**docker ps**" para ver os containers em execução, veremos que o container ***cont_servico_assinatura*** está sendo executado na porta 5001:

```
PS C:\Users\kelvi> docker ps
CONTAINER ID   IMAGE                 COMMAND                  CREATED         STATUS         PORTS                                          NAMES
5916686acfcf   servico_assinatura    "flask run --host=0.…"   3 minutes ago   Up 3 minutes   0.0.0.0:5001->5001/tcp, [::]:5001->5001/tcp    cont_servico_assinatura
PS C:\Users\kelvi>
```

2. Criando o container do serviço servico_catalogo

```
docker run -d -p 5002:5002 --network netflix-rede --name
cont_servico_catalogo servico_catalogo
```

```
PS C:\Users\kelvi> docker run -d -p 5002:5002 --network netflix-rede --name cont_servico_catalogo servico_catalogo
4f6d7678c666b3319a48bdd9db82f4b62c1b705b60b4fb98a2a08549d4f69109
PS C:\Users\kelvi>
```

```
PS C:\Users\kelvi> docker ps
CONTAINER ID   IMAGE                 COMMAND                  CREATED         STATUS         PORTS                                          NAMES
4f6d7678c666   servico_catalogo      "flask run --host=0.…"   3 minutes ago   Up 2 seconds   0.0.0.0:5002->5002/tcp, [::]:5002->5002/tcp    cont_servico_catalogo
5916686acfcf   servico_assinatura    "flask run --host=0.…"   4 minutes ago   Up 4 minutes   0.0.0.0:5001->5001/tcp, [::]:5001->5001/tcp    cont_servico_assinatura
PS C:\Users\kelvi>
```

3. Criando o container do serviço mine_netflix

```
docker run -d -p 5000:5000 --network netflix-rede --name
cont_mine_netflix mine_netflix
```

```
PS C:\Users\kelvi> docker run -d -p 5000:5000 --network netflix-rede --name cont_mine_netflix mine_netflix
114076d83d4d5ddd4e25ce0f2b18103ce302db9326eb0c3e37dea7d9d841fd27
PS C:\Users\kelvi>
```

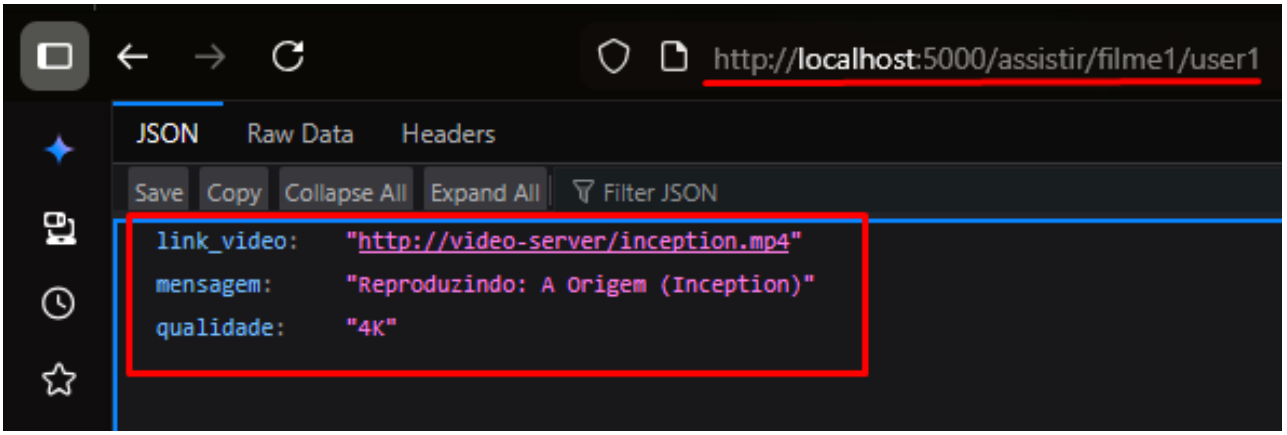
```
PS C:\Users\kelvi> docker ps
CONTAINER ID   IMAGE              COMMAND                  CREATED         STATUS          PORTS                                          NAMES
114076d83d4d   mine_netflix       "flask run --host=0.…"   3 minutes ago   Up 1 second     0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp    cont_mine_netflix
4f6d7678c666   servico_catalogo   "flask run --host=0.…"   4 minutes ago   Up 45 seconds   0.0.0.0:5002->5002/tcp, [::]:5002->5002/tcp    cont_servico_catalogo
5916686acfcf   servico_assinatura "flask run --host=0.…"   5 minutes ago   Up 5 minutes    0.0.0.0:5001->5001/tcp, [::]:5001->5001/tcp    cont_servico_assinatura
```

---

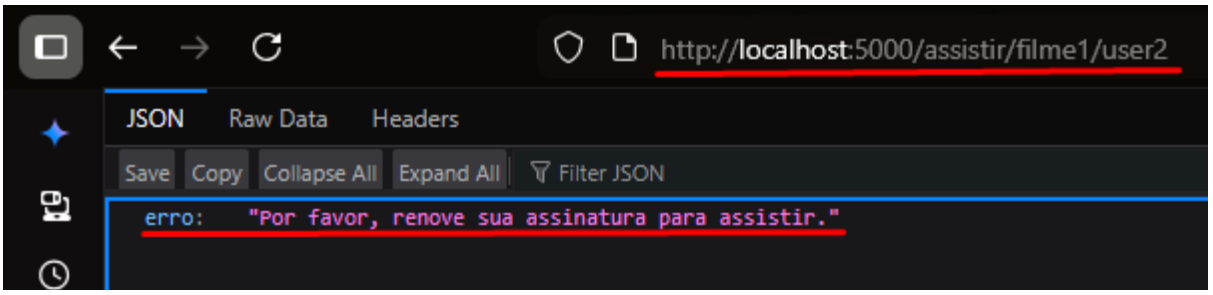
7. Vamos verificar se o sistema está funcionando:

Abra o navegador e digite: http://localhost:5000/assistir/filme1/user1

Esse URL está pedindo ao serviço **mine_netflix** que está rodando na porta **5000** o **filme1** para o **user1**. Receberemos um json como na imagem abaixo:



Mas se digitarmos o URL http://localhost:5000/assistir/filme1/user2, ou seja, pedir o filme1 para o user2, receberemos:



Comandos Úteis de Gerenciamento

Aqui está um "Guia Rápido" (Cheat Sheet) do laboratório Mini-Netflix:

Etapa	Ação / Objetivo	Comando
1. Rede	**Criar a rede** para os contêineres conversarem	docker network create netflix-rede
	Verificar se a rede foi criada	docker network ls
2. Build	Criar imagem do **Gateway** (dentro da pasta mine_netflix)	docker build -t mine_netflix .
(Construção)	Criar imagem da **Assinatura** (dentro da pasta servico_assinatura)	docker build -t servico_assinatura .
	Criar imagem do **Catálogo** (dentro da pasta servico_catalogo)	docker build -t servico_catalogo .
	Verificar todas as imagens criadas	docker images

3. Run	Rodar Container de **Assinatura** (Porta 5001)	docker run -d -p 5001:5001 --network netflix-rede --name cont_servico_assinatura servico_assinatura
(Execução)	Rodar Container de **Catálogo** (Porta 5002)	docker run -d -p 5002:5002 --network netflix-rede --name cont_servico_catalogo servico_catalogo
	Rodar Container do **Gateway** (Porta 5000)	docker run -d -p 5000:5000 --network netflix-rede --name cont_mine_netflix mine_netflix
4. Gestão	Ver contêineres rodando no momento	docker ps
	Ver todos os contêineres (incluindo os parados/erro)	docker ps -a
	Ver logs de erro de um contêiner específico	docker logs nome-do-conteiner
	Parar um contêiner	docker stop nome-do-conteiner